```
      val behavior: Behavior[String] =                    ❶
        Behaviors.stopped

      val carl = spawn(behavior, "carl")                  ❷

      LoggingTestKit.empty                                ❸
        .withLogLevel(Level.INFO)                         ❹
        .withMessageRegex(                                ❹
          ".*Message.*to.*carl.*was not delivered.*2.*dead letters encountered")
        .expect {
          carl ! "Hello"
          carl ! "Hello"
        }
    }
}
```

❶ Behavior that stopped at instantiation

❷ ScalaTestWithActorTestKit extends ActorTestKitBase, which has .spawn and more methods

❸ Constructor

❹ Adds attributes to the filter

This simple example proves that two messages were sent to `deadLetters` instead of the original `carl` actor.

In this section, you have seen how asynchronous testing is similar to synchronous testing. However, there are some important differences, such as the creation of the `ActorSystem`, testing real actors, and the ability to schedule messages. Next, let's look at some unique features of the `ActorTestKit`.

## *4.3.4 Log capturing*

Logging is essential to any application, but reading the logs generally is not useful when testing—at least, not if all the tests are successful. Log statements become noise in these circumstances.